

Ethan Patterson

HW1- Problem 3

a.) The algorithm is a solution to the plant pot resizer problem that uses heapsort with a max heap to quickly load and fetch the max value for both the plants and the pots. Then, it checks if the biggest remaining plant can fit in the biggest pot available, if so, puts the (now empty) old pot back in the pot heap, and checks the next biggest plant + pot combo. If the biggest pot is smaller or equal to the current plant size, it fails. Otherwise it succeeds when the plant heap is empty.

b.) pseudo code:

Note: this varies from the actual code because the recursion depth needed forces a segfault for one case in the autograder (but not on my pc) and I had to use a loop rather than the recursive technique. I'm not particularly interested in rewriting the pseudocode as well after having to figure out that was the problem. The algorithm is functionally intact either way with no change in time complexity or core logic.

```
main{
  Plants[]
  Planters[]
  Add all plants to plant heap
  Add all pots to pot heap
  plantMover(plants, planters)
}

plantMover(plants[], planters[]){
  If plant.size == 0 return true

  biggestPlant= getMax(plants)
  biggestPot = getMax(planters)

  If biggestPot <= biggestPlant return false

  planters.add(biggestPlant)

  return plantMover(plants, planters)
}
```

c.) The best fit for the biggest plant left will always be the biggest pot left, as its pot can always be used for a smaller plant. Thus, as long as we pull the biggest plant out and the biggest pot out one by one, leaving the leftover pot from the swap, we'll always ensure we can (or cant) swap every plant.

d.) $O(n \log(n))$

e.) It's heapsort. Adds N (being $r+p$) elements, and an add for a heap takes $\log n$. Extracts are also $O(\log(n))$ which we perform $2p$ times, which is just going to collapse back into the original $O(n \log(n))$ estimate.